

CY Cergy-Paris Université

RAPPORT

pour le projet Génie Logiciel
L2 Informatique, 2025-2026

sur le sujet

Tu n'y peux rien

rédigé par

KHODJA Laiza
BOUATMANE Nesrine
BERRANDOU Nassim

Table des matières

Remerciements	2
1 Introduction	3
1.1 Contexte du projet	3
1.2 Objectif du projet	3
1.3 Organisation du rapport	3
2 Spécification du projet	4
2.1 Notions de base et contraintes du projet	4
2.1.1 Fonctionnement général du logiciel	4
2.1.2 Notions et terminologies de base	4
2.1.3 Contraintes et limitations connues	4
2.2 Fonctionnalités attendues du projet	4
3 Conception et mise en œuvre	5
3.1 Architecture globale du logiciel	5
3.2 Classes de données (engine.data)	5
3.3 Traitements (engine.process)	5
3.4 IHM Graphique (gui)	5
3.5 Patterns de conception utilisés	5
4 Manuel utilisateur	6
4.1 Lancement du jeu	6
4.2 Configuration de la partie	6
4.3 Déroulement d'une partie	7
4.4 Fin de partie	7
5 Déroulement du projet	8
5.1 Réalisation du projet par étapes	8
5.2 Répartition des tâches, communication et intégration	8
6 Conclusion et perspectives	9
6.1 Résumé du travail réalisé	9
6.2 Améliorations possibles du projet	9

Table des figures

1	Architecture du logiciel	5
2	Diagramme de classes UML	6
3	Menu principal	6
4	Écran des options	7
5	Interface de jeu	7
6	Écran de fin de partie	8

Liste des tableaux

Remerciements

Les auteurs du projet voudraient remercier vivement :

- M. Tianxiao LIU, enseignant du module Génie Logiciel, pour ses conseils et son accompagnement tout au long du projet.
- CY Cergy Paris Université pour nous avoir fourni les ressources nécessaires à la réalisation de ce travail.
- L'équipe pédagogique de la Licence Informatique pour la qualité de la formation.

1 Introduction

1.1 Contexte du projet

Le projet "Tu n'y peux rien" s'inscrit dans le cadre de l'UE Génie Logiciel de deuxième année à CY Cergy Paris Université. L'objectif est de développer un jeu de cartes multijoueur en Java, respectant des règles spécifiques et intégrant des bots de différents niveaux de difficulté.

1.2 Objectif du projet

Créer un jeu de cartes où un joueur humain affronte 2 à 4 bots. Le jeu doit gérer les combinaisons (simple, double, série, bombe, double joker), les cartes spéciales (2, Joker), et calculer les scores en fin de partie.

1.3 Organisation du rapport

La section 2 présente le cahier des charges et les fonctionnalités attendues. La section 3 détaille la conception et l'architecture du logiciel. La section 4 est un manuel utilisateur. La section 5 décrit le déroulement du projet. La section 6 conclut et propose des améliorations.

2 Spécification du projet

2.1 Notions de base et contraintes du projet

2.1.1 Fonctionnement général du logiciel

Le jeu se déroule en tours. Chaque joueur reçoit 5 cartes au début. Le joueur avec la plus petite carte (hors 2 et Joker) commence. À chaque tour, le joueur doit poser une combinaison qui bat la précédente. Si aucun joueur ne peut jouer, le tas est réinitialisé.

2.1.2 Notions et terminologies de base

- **Carte** : 54 cartes (52 standards + 2 Jokers)
- **Combinaison simple** : 1 carte
- **Double** : 2 cartes identiques
- **Série** : 3 cartes consécutives
- **Bombe** : 3 ou 4 cartes identiques
- **Double Joker** : 2 Jokers ensemble (imbattable)

2.1.3 Contraintes et limitations connues

- Langage : Java 8
- Interface graphique : Swing
- Bibliothèques externes : Log4j (logs), JUnit (tests)
- La musique ne fonctionne que sous Windows (bug Linux)

2.2 Fonctionnalités attendues du projet

Fonctionnalités implémentées :

- Distribution de 5 cartes à chaque joueur
- Détection automatique du joueur qui commence
- Pose de combinaisons (simple, double, série, bombe, double joker)
- Gestion des cartes spéciales (2 écrase tout, joker polymorphe)
- Pioche automatique si aucune carte jouable
- Reset du tour quand tous les joueurs passent
- Recyclage du deck quand vide
- Interface graphique complète (menu, options, jeu, fin de partie)
- 3 niveaux de difficulté pour les bots (Facile, Moyen, Difficile)
- Système de scores (1 carte = 1 point, bombe = $\times 2$, double joker = $\times 4$)
- Logs détaillés (texte et HTML)
- Tests unitaires JUnit

3 Conception et mise en œuvre

3.1 Architecture globale du logiciel

L'architecture du projet suit le modèle MVC (Modèle-Vue-Contrôleur) :

- **Modèle (engine.data)** : Classes Card, Deck, Player, GameStats
- **Contrôleur (engine.process)** : Classes Game, Combination, BotPlayer
- **Vue (gui)** : Classes MainGUI, MenuGUI, OptionGUI, EndGameGUI

FIGURE 1 – Architecture du logiciel

3.2 Classes de données (engine.data)

- **Card** : Représente une carte avec sa valeur et sa couleur. Implémente equals() et hashCode().
- **Deck** : Gère le paquet de 54 cartes, mélange (shuffle) et pioche (draw).
- **Player** : Gère la main du joueur, la pioche et la pose de combinaisons.
- **CombinationType** : Énumération des types (SIMPLE, DOUBLE, SERIES, BOMB, DOUBLE_JOKER, INVALID).
- **GameStats** : Enregistre les statistiques et calcule les scores.

3.3 Traitements (engine.process)

- **Combination** : Détection automatique du type avec determineType(), comparaison avec canBeat().
- **Game** : Logique principale (gestion des tours, pioche, reset, recyclage).
- **BotPlayer** : Classe abstraite avec stratégies différentes.
- **EasyBot** : Joue aléatoirement, 30% de chance de passer.
- **MediumBot** : Évite les bombes, joue les combinaisons faibles.
- **HardBot** : Stratégique, privilégie les combinaisons fortes.

3.4 IHM Graphique (gui)

L'interface est organisée en trois sous-dossiers :

- **screens/** : Écrans complets (MainGUI, MenuGUI, OptionGUI, EndGameGUI)
- **panels/** : Composants réutilisables (CardPanel, BotsPanel, DeckPanel, GameInfoBar, GameLogPanel)
- **utils/** : Utilitaires (GameStyle, CardPaintStrategy, MusicPlayer)

3.5 Patterns de conception utilisés

- **Héritage** : BotPlayer → EasyBot, MediumBot, HardBot
- **Factory** : GameBuilder.createBot()
- **MVC implicite** : Séparation data/process/gui
- **Stratégie** : Comportements différents des bots
- **Singleton implicite** : GameStyle.isFullscreen

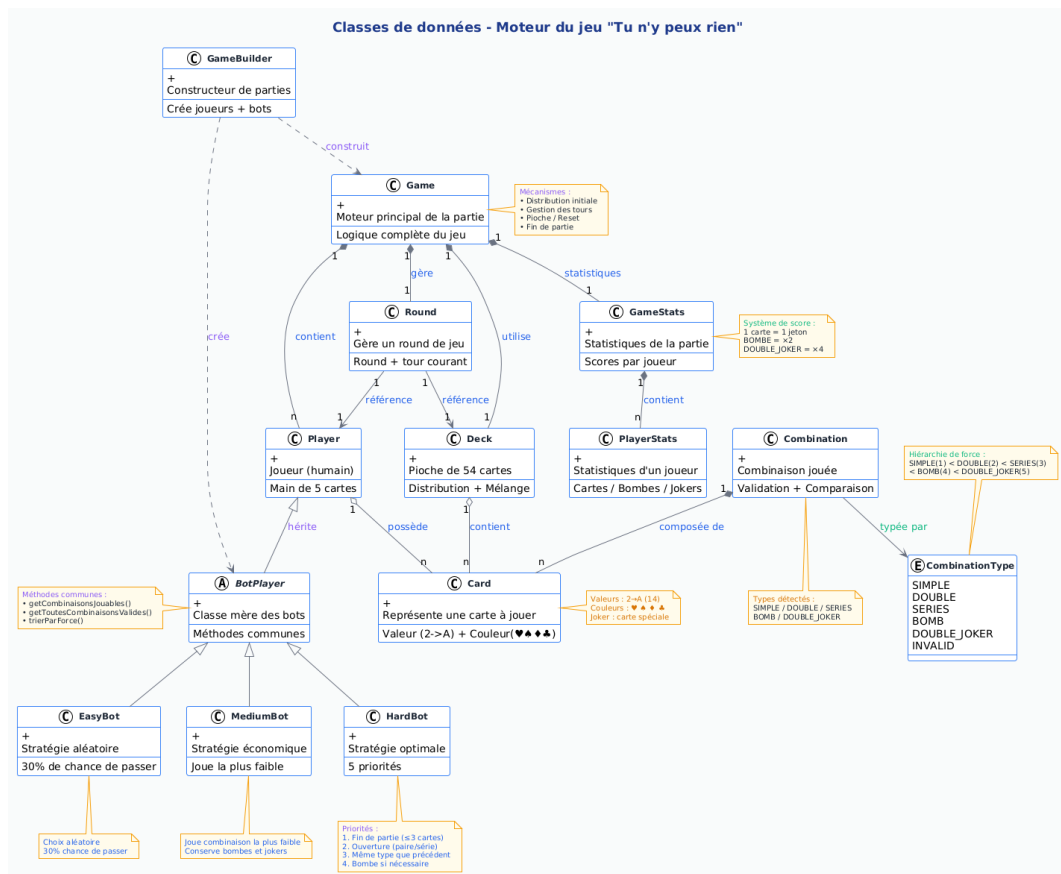


FIGURE 2 – Diagramme de classes UML

4 Manuel utilisateur

4.1 Lancement du jeu

Exécutez la classe `test.TestGame`. Le menu principal apparaît.



FIGURE 3 – Menu principal

4.2 Configuration de la partie

Cliquez sur "Nouvelle partie" pour accéder aux options. Sélectionnez le nombre de joueurs (3 à 5) et la difficulté des bots (Facile, Moyen, Difficile).



FIGURE 4 – Écran des options

4.3 Déroulement d'une partie

Le joueur humain est "Vous". Les bots sont représentés par des avatars. Pour jouer, sélectionnez des cartes dans votre main (clic gauche) puis cliquez sur "JOUER LES CARTES SÉLECTIONNÉES". Si vous ne pouvez pas jouer, cliquez sur "PIOCHER".

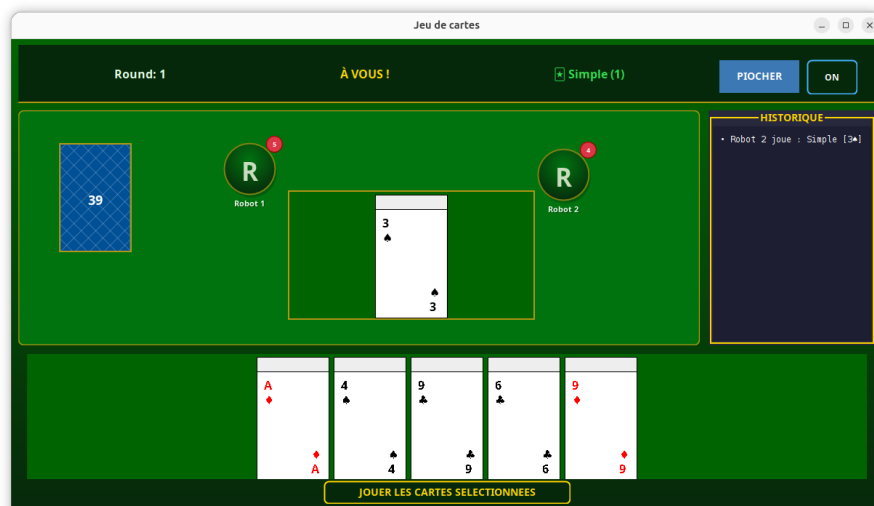


FIGURE 5 – Interface de jeu

4.4 Fin de partie

Quand un joueur n'a plus de cartes, l'écran de fin s'affiche avec le podium, les scores et les statistiques.



FIGURE 6 – Écran de fin de partie

5 Déroulement du projet

5.1 Réalisation du projet par étapes

Le projet a été réalisé selon les étapes suivantes :

1. **Semaine 1-2** : Mise en place des classes de base (Card, Deck, Player)
2. **Semaine 3-4** : Implémentation des combinaisons (Combination, determineType, canBeat)
3. **Semaine 5-6** : Développement des bots et de la logique de jeu (Game)
4. **Semaine 7-8** : Interface graphique (Swing) et refonte du layout
5. **Semaine 9-10** : Tests unitaires (JUnit) et corrections de bugs
6. **Semaine 11-12** : Rédaction du rapport et préparation de la livraison

5.2 Répartition des tâches, communication et intégration

La répartition des tâches s'est organisée comme suit :

- **Laiza KHODJA** : Interface graphique (GameStyle, MusicPlayer, EndGameGUI, BotsPanel)
- **Nesrine BOUATMANE** : Classes de données (Card, Deck, Player, GameStats, tests unitaires)
- **Nassim BERRANDOU** : Logique métier (Game, Combination, bots, correction bugs)

La communication s'est faite via Discord et GitHub. Le versionnement Git a permis une intégration continue des différentes parties.

Les principaux défis rencontrés ont été :

- La gestion du reset du tas et du recyclage des cartes
- La compatibilité de la musique entre Linux et Windows
- La synchronisation des tours entre le joueur humain et les bots

6 Conclusion et perspectives

6.1 Résumé du travail réalisé

Le projet "Tu n'y peux rien" a été mené à bien dans les délais impartis. Toutes les fonctionnalités demandées dans le cahier des charges ont été implémentées :

- Jeu complet avec interface graphique Swing
- 3 niveaux de difficulté pour les bots (Facile, Moyen, Difficile)
- Détection des combinaisons (simple, double, série, bombe, double joker)
- Gestion des cartes spéciales (2, Joker)
- Système de scores et statistiques
- Tests unitaires (JUnit) couvrant les classes principales
- Logs détaillés (Log4j)

Au total, ce sont plus de 40 classes Java qui ont été développées, avec une couverture de tests satisfaisante sur les composants critiques.

6.2 Améliorations possibles du projet

Plusieurs extensions pourraient être apportées au projet :

- **Mode multijoueur en ligne** : Permettre à plusieurs joueurs humains de jouer à distance via réseau
- **Sauvegarde des parties** : Sauvegarder l'état du jeu pour le reprendre plus tard
- **Classements et statistiques persistants** : Stocker les scores et victoires dans un fichier ou une base de données
- **Animations** : Ajouter des animations lors de la pose des cartes
- **Thèmes personnalisables** : Permettre au joueur de changer l'apparence des cartes et du tapis
- **Version mobile** : Adapter le jeu pour Android/iOS

Ces améliorations pourraient faire l'objet d'une future version du projet.

Références

- [1] Apache log4j. <https://logging.apache.org/log4j/1.2/>, 2024.
- [2] Java swing documentation. <https://docs.oracle.com/javase/8/docs/technotes/guides/swing/>, 2024.
- [3] Junit 4. <https://junit.org/junit4/>, 2024.
- [4] L. M. Haas, E. T. Lin, and M. A. Roth. Data integration through database federation. *IBM Syst. J.*, 41(4) :578–596, 2002.
- [5] John L. Hennessy and David A. Patterson. *Computer Architecture, Fifth Edition : A Quantitative Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 5th edition, 2011.